

Université de Sfax

Institut Supérieur d'Informatique
et de Multimédia de Sfax

MÉMOIRE

Présenté à

*L'Institut Supérieur d'Informatique
et de Multimédia de Sfax*

En vue de la validation de

DIPLÔME NATIONAL D'INGÉNIEUR EN INFORMATIQUE

Intitulé

***Système intelligent
de suivi de livraison***

élaboré par

Mohamed Aziz Hadjkacem

Soutenu le XX/06/2026 devant le jury composé de

M. XXX

M. XXX

M. Bassem Bouaziz

M. XXX

Président

Rapporteur

Encadrant académique

Encadrant professionnel

Dédicace

Je dédie ce travail, fruit de plusieurs années d'efforts et d'apprentissage, à toutes les personnes qui m'ont accompagné jusqu'ici.

Je dédie ce travail à mes parents pour leur soutien constant, leur patience et les sacrifices qu'ils ont faits pour me permettre d'arriver là où je suis aujourd'hui. Sans eux, rien de tout cela n'aurait été possible.

À ma famille, pour ses encouragements et sa présence à chaque étape de ce chemin.

À mes amis, pour les bons moments partagés, pour leur aide et leur soutien dans les périodes difficiles comme dans les bonnes.

À mes professeurs, à l'ISIMS et auparavant, pour les connaissances qu'ils m'ont transmises et qui ont posé les bases de ce travail.

Enfin, je dédie ce mémoire à moi-même, pour la persévérance et le travail qu'il a représenté tout au long du projet.

Mohamed Aziz

■ Remerciements

La réalisation de ce projet de fin d'études n'aurait pas été possible sans le soutien, la confiance et l'accompagnement de plusieurs personnes, à qui j'exprime ma profonde gratitude.

À mon encadrant académique, **M. Bassem Bouaziz**, pour la qualité de son encadrement, la pertinence de ses conseils et sa disponibilité tout au long de ce travail. Son exigence et sa rigueur ont été une véritable source de progression.

À mon encadrant professionnel, [Nom à compléter], pour son accompagnement et ses orientations techniques tout au long de ce projet.

À la direction et aux équipes d'**ASM (All Soft Multimedia)**, pour m'avoir accueilli dans un environnement de travail stimulant, pour la confiance accordée et pour les moyens mis à ma disposition tout au long de cette expérience.

Aux membres du jury, pour l'honneur qu'ils me font en évaluant ce travail, ainsi que pour l'intérêt et les remarques qui contribueront à son amélioration.

Mes remerciements vont également à l'ensemble du corps enseignant de l'**ISIMS**, pour la formation solide et les compétences qui ont constitué la base de ce projet et de mon parcours académique.

Enfin, j'exprime ma gratitude à toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail.



Table des matières

Dédicace	i
Remerciements	ii
Liste des abréviations	viii
INTRODUCTION GÉNÉRALE	1
1 Cadre général du projet	3
1.1 Introduction	3
1.2 Présentation de l’organisme d’accueil	3
1.3 Présentation du projet	4
1.3.1 Contexte du projet	4
1.3.2 Problématique	4
1.3.3 Objectifs du projet	4
1.3.4 Périmètre du projet	5
1.4 Étude et critique de l’existant	5
1.4.1 Aperçu des outils existants	5
1.4.2 Synthèse comparative et solution proposée	6
1.5 Méthodologie de développement	8
1.5.1 Présentation de Scrum	8
1.5.2 Organisation du travail et outils	8
1.6 Conclusion	9
2 Sprint 0 — Initialisation, besoins et étude technique	10
2.1 Introduction	10
2.2 Identification des acteurs	10
2.3 Analyse et spécification des besoins	12
2.3.1 Besoins fonctionnels	12
2.3.2 Besoins non fonctionnels	13
2.3.3 Diagramme global des cas d’utilisation	14
2.3.4 Diagramme de classes global	15

2.4	Gestion du projet avec Scrum	17
2.4.1	Product Backlog	17
2.4.2	Planification des sprints	21
2.5	Étude technique	22
2.5.1	Environnement matériel	22
2.5.2	Environnement logiciel de développement	23
2.5.3	Technologies retenues	23
2.6	Conclusion	24
3	Architecture et conception générale	26
3.1	Introduction	26
3.2	Une application unique ou plusieurs services	26
3.3	Vue d'ensemble	28
3.3.1	La passerelle	29
3.3.2	Les services	30
3.3.3	Les composants d'infrastructure	30
3.4	Du modèle de classes aux bases de données	31
3.4.1	Ce que devient une relation entre deux services	32
3.4.2	L'évolution du schéma	32
3.5	L'isolation des entreprises clientes	32
3.5.1	Trois façons de séparer, une seule retenue	33
3.5.2	Le trajet d'une requête	33
3.5.3	Le refus plutôt que le repli	34
3.5.4	Le cas des traitements asynchrones	34
3.6	Les échanges entre services	35
3.6.1	Quand un message ne passe pas	36
3.7	La mise en marche	36
3.8	Conclusion	37
4	Sprint 1 — Socle d'identité, d'autorisation et de multi-tenance	38
4.1	Introduction	38
4.2	Conclusion	38
5	Sprint 2 — Intégration ERP agnostique	39
5.1	Introduction	39
5.2	Conclusion	39
6	Sprint 3 — Planification, exécution et suivi temps réel	40
6.1	Introduction	40
6.2	Conclusion	40

7	Sprint 4 — Application mobile chauffeur et mode hors ligne	41
7.1	Introduction	41
7.2	Conclusion	41
8	Sprint 5 — Exploitation, résilience et assistant interne	42
8.1	Introduction	42
8.2	Conclusion	42
9	Validation globale, qualité et déploiement	43
9.1	Introduction	43
9.2	Conclusion	43
	Conclusion générale et perspectives	44
	Bibliographie et webographie	45
A	Annexes	46



Table des figures

1.1	Logo de l'entreprise ASM (All Soft Multimedia) [ASM]	3
1.2	Logo d'Onfleet [Onfa]	5
1.3	Logo de Bringg [Bria]	6
2.1	Diagramme global des cas d'utilisation	14
2.2	Diagramme de classes global	16
3.1	Vue d'ensemble de l'architecture	29
3.2	Répartition des données entre les services	31
3.3	Le choix du schéma, de la requête jusqu'à la base	33
3.4	Topologie de la messagerie	35



Liste des tableaux

- 1.1 Fiche d’identité d’ASM 4
- 1.2 Comparatif des solutions existantes, d’après leur documentation publique . . . 7
- 1.3 Comparaison des méthodologies de développement 8

- 2.1 Acteurs du système et périmètre de chacun 11
- 2.2 Besoins non fonctionnels 13
- 2.3 Product Backlog du projet 17
- 2.4 Planification des sprints 22
- 2.5 Environnement matériel de développement 23
- 2.6 Outils de développement et de collaboration 23
- 2.7 Technologies retenues et motif du choix 24

- 3.1 Application unique ou services séparés 26
- 3.2 Les services et leurs responsabilités 30
- 3.3 Les conteneurs de l’installation 36

Liste des abréviations

API	Application Programming Interface
ERP	Enterprise Resource Planning
JWT	JSON Web Token
OIDC	OpenID Connect
RBAC	Role-Based Access Control
SaaS	Software as a Service
SLA	Service Level Agreement
UML	Unified Modeling Language

INTRODUCTION GÉNÉRALE

Dans une entreprise de distribution, la livraison ne s'arrête pas à la sortie de l'entrepôt : elle continue jusqu'à la porte du client, souvent hors de portée du système d'information. Un dispatcheur doit affecter des commandes à des livreurs, suivre leur avancement, réagir à un incident, tout en gardant l'ERP — Odoo, ERPNext ou un autre — synchronisé avec ce qui se passe réellement sur le terrain. Ce dernier maillon, le *dernier kilomètre*, est celui où l'information se perd le plus facilement : un bon de livraison papier égaré, un stock qui ne se met pas à jour, une preuve de remise introuvable en cas de litige.

Les outils du marché répondent rarement à ce problème dans son ensemble. Un logiciel de tournées générique n'a pas connaissance des commandes de l'ERP et impose de les ressaisir ; un module fourni par l'éditeur de l'ERP enferme, à l'inverse, l'entreprise dans cet éditeur précis et ne suit plus si elle en change. Et la plupart de ces outils supposent une connexion permanente, alors qu'un livreur en zone rurale ou en sous-sol de parking n'en a pas toujours une.

Ce projet de fin d'études, mené au sein de l'entreprise **ASM (All Soft Multimedia)**, elle-même éditrice d'un progiciel de gestion, part de ce constat. Le travail réalisé, un **Système intelligent de suivi de livraison**, repose sur deux partis pris structurants : un adaptateur qui dialogue avec l'ERP par un contrat commun plutôt que par un développement propre à chacun, et une isolation des données par entreprise cliente qui permet à une seule plateforme de servir plusieurs d'entre elles. Autour de ce socle s'organisent la planification des tournées, le suivi en temps réel et une application chauffeur qui reste utilisable sans réseau. Le système couvre ainsi tout le trajet d'une commande : de son import depuis l'ERP jusqu'à la preuve de livraison qui y retourne.

Ce rapport retrace la démarche suivie pour construire ce système, chapitre après chapitre :

- **Chapitre 1** : présente le cadre général du projet, l'entreprise d'accueil et l'étude de l'existant.
- **Chapitre 2** : présente le sprint 0, consacré à l'analyse des besoins, à la planification Scrum et à l'étude technique.
- **Chapitre 3** : détaille l'architecture et la conception générale du système.

- **Chapitre 4** : détaille le sprint 1, consacré au socle d'identité, d'autorisation et aux référentiels d'exploitation.
- **Chapitre 5** : décrit le sprint 2, consacré à l'intégration ERP agnostique.
- **Chapitre 6** : expose le sprint 3, consacré à la planification des tournées et à leur suivi en temps réel.
- **Chapitre 7** : présente le sprint 4, consacré à l'application mobile chauffeur et au mode hors ligne.
- **Chapitre 8** : présente le sprint 5, consacré à l'exploitation, à la résilience et à l'assistant interne.
- **Chapitre 9** : présente la validation globale, la qualité et le déploiement du système.

Une conclusion générale reviendra, à la fin de ce rapport, sur ce qui a été construit et sur ce qui reste à faire évoluer.

1.1 Introduction

Ce chapitre pose le cadre dans lequel s'inscrit ce projet de fin d'études. Nous présentons d'abord l'entreprise d'accueil, puis le contexte, la problématique, les objectifs et le périmètre du projet. Nous analysons ensuite les solutions existantes afin de situer notre approche par rapport à elles. Nous terminons par la méthodologie de développement retenue.

1.2 Présentation de l'organisme d'accueil

Ce projet a été réalisé au sein de l'entreprise **ASM (All Soft Multimedia)**, une société de services numériques tunisienne fondée en 2007. Éditrice et intégratrice de solutions logicielles, ASM conçoit des sites web, des applications mobiles, des systèmes de point de vente ainsi qu'un progiciel de gestion intégré propriétaire, **DUX**, distribué auprès de plus de 4500 clients. Son siège se trouve à Sfax, avec une présence à Sousse et Tunis ainsi que des filiales à Alger et à Dakar [ASM].



FIG. 1.1 : Logo de l'entreprise ASM (All Soft Multimedia) [ASM]

Ce positionnement est directement pertinent pour ce projet : ASM n'est pas seulement le commanditaire, elle est elle-même éditrice d'un ERP. Une plateforme de suivi de livraison qui s'intègre à un ERP sans s'y enfermer profite à ses clients quel que soit le progiciel qu'ils exploitent, et son architecture — un contrat d'adaptateur commun à tout ERP — est conçue pour qu'un autre système puisse s'y brancher sans remettre en cause le reste de la plateforme.

TAB. 1.1 : Fiche d'identité d'ASM

Nom de l'entreprise	ASM (All Soft Multimedia)
Secteur d'activité	Édition et intégration de logiciels : ERP (DUX), sites web, applications mobiles, systèmes de point de vente
Date de création	2007
Siège social	Sfax, Tunisie
Présence	Sfax, Sousse, Tunis (Tunisie); Alger (Algérie); Dakar (Sénégal)
Site web	https://asm-tunisie.com

1.3 Présentation du projet

1.3.1 Contexte du projet

Un ERP, quel qu'il soit, gère efficacement les commandes, les stocks et la facturation, mais s'arrête à la porte de l'entrepôt. Ce qui se passe ensuite — l'affectation d'un livreur, le suivi de sa tournée, la collecte de la preuve de remise, un éventuel retour — reste hors de sa portée, ou suppose de développer un module spécifique à chaque ERP concerné. Or les entreprises que sert ASM n'exploitent pas toutes le même système, et ce choix ne devrait pas déterminer si elles peuvent, ou non, suivre leurs livraisons.

1.3.2 Problématique

La problématique de ce projet peut être résumée ainsi :

Comment concevoir une plateforme de suivi de livraison exploitable par plusieurs entreprises clientes, indépendante de l'ERP utilisé par chacune, et capable de fonctionner sur le terrain même en l'absence de connexion réseau ?

1.3.3 Objectifs du projet

L'objectif principal est de couvrir l'ensemble du cycle de vie d'une livraison, depuis l'import de la commande jusqu'à la preuve de sa remise, à travers :

- une architecture **multi-tenant**, où chaque entreprise cliente dispose de ses propres données isolées au sein d'une même plateforme ;
- une intégration ERP **agnostique**, opérationnelle sur Odoo et ERPNext par un adaptateur commun plutôt qu'un développement spécifique à chacun, et conçue pour qu'un autre ERP puisse s'y brancher sans toucher au reste du système ;

- une application mobile chauffeur fonctionnant en **mode hors ligne**, pour que la perte de réseau sur le terrain n’interrompe pas la livraison ;
- un suivi en **temps réel** des tournées, de la planification jusqu’à la remise au client final ;
- une **chaîne d’encaissement** traitée comme un circuit de responsabilité, de la collecte des espèces à la porte du client jusqu’à leur remise au dépôt et l’arbitrage des écarts ;
- un **assistant interne** auquel un opérateur pose ses questions en langage naturel, et qui répond sur l’activité de son entreprise sans jamais lui donner accès à ce que son rôle ne l’autorise pas à consulter.

1.3.4 Périmètre du projet

Le projet couvre le cycle complet d’une livraison — import, planification de tournée, exécution terrain, preuve de livraison, encaissement et retours — pour un nombre quelconque d’entreprises clientes. Il se limite au dernier kilomètre : la gestion des stocks, la facturation et la production restent du ressort de l’ERP, que la plateforme consulte et met à jour sans s’y substituer.

1.4 Étude et critique de l’existant

1.4.1 Aperçu des outils existants

Avant de concevoir la solution, nous avons examiné les outils déjà utilisés pour le suivi de livraison du dernier kilomètre.



FIG. 1.2 : Logo d’Onfleet [Onfa]

Onfleet [Onfa] est une plateforme SaaS de gestion de tournées et de suivi de chauffeurs, reconnue pour son optimisation d’itinéraires et son suivi en temps réel. Elle ne s’intègre toutefois à un ERP que par des connecteurs génériques ou un développement sur mesure. Elle propose un mode hors ligne, mais désactivé par défaut : sans lui, un chauffeur ne dispose que de deux minutes pour démarrer ou terminer une tâche avant d’être bloqué faute de réseau [Onfb]. Sa documentation publique ne décrit pas de mécanisme d’encaissement en espèces ni de remise de caisse.



FIG. 1.3 : Logo de Bringg [Bria]

Bringg [Bria] est une plateforme de logistique de livraison orientée grands comptes, proposant orchestration de flotte et visibilité client. Son application chauffeur reste utilisable hors ligne pour les tâches déjà assignées [Brib], et son API prévoit la prise d'un paiement par le livreur ainsi qu'un montant restant dû sur la commande [Bric]. Sa richesse fonctionnelle a pour contrepartie une mise en œuvre lourde, pensée pour un déploiement par ERP ou canal de vente plutôt que pour une intégration légère et multi-ERP.

Le module de livraison natif d'un ERP (par exemple Odoo Inventory) constitue la troisième approche possible. Il a l'avantage d'être déjà connecté aux données de commande et de stock, mais il enferme l'entreprise dans cet ERP précis : le changer, ou en exploiter plusieurs à la fois — un cas concret chez les clients d'ASM — n'est pas pris en charge.

1.4.2 Synthèse comparative et solution proposée

Le fonctionnement hors connexion, longtemps distinctif, est aujourd'hui proposé par les plateformes matures du marché. Ce qui les sépare de notre approche tient à deux points : la profondeur de l'intégration ERP, générique chez elles alors que le projet vise un contrat commun réellement implémenté sur plusieurs ERP, et le traitement de l'encaissement à la livraison. Sur ce second point, la distinction porte moins sur la collecte du paiement — Bringg documente une action chauffeur de prise de paiement et un montant restant dû — que sur ce qui vient après : la remise de la caisse du livreur, son comptage contradictoire et l'arbitrage des écarts. Ce maillon est déterminant sur le marché tunisien, où le paiement en espèces à la remise reste le mode dominant.

Un troisième écart tient à la nature de l'assistance intégrée. Onfleet a doté son tableau de bord d'un assistant conversationnel en avril 2026, inclus dans tous ses abonnements ; sa documentation le présente comme une aide à l'usage du produit : il répond aux questions de prise en main et de dépannage [Onfc]. Bringg, de son côté, ne documente aucun assistant conversationnel et concentre son intelligence artificielle sur la prédiction et l'optimisation des tournées [Brid]. Aucun des deux ne documente donc un assistant capable d'interroger les données d'exploitation du client en langage naturel.

C'est précisément la fonction retenue ici, avec deux garde-fous qui en font autre chose qu'un agent conversationnel branché sur une base. D'abord, l'assistant **hérite des droits de celui qui**

l'interroge : il ne dispose d'aucun compte propre, et deux utilisateurs de rôles différents n'obtiennent pas la même réponse à la même question. Ensuite, il **ne peut que lire** : aucune de ses opérations ne modifie l'état du système. Il cite enfin ses sources, de sorte que chaque réponse reste vérifiable par celui qui la reçoit.

TAB. 1.2 : Comparatif des solutions existantes, d'après leur documentation publique

Solution	Multi-ERP	Versions ERP	Remise de caisse	Hors ligne	Assistant sur les données	Limite principale
Onfleet	Partiel	Sans objet	Non	Option	Support produit	Intégration ERP superficielle
Bringg	Partiel	Sans objet	Partiel	Oui	Non documenté	Mise en œuvre lourde, pensée par ERP
Module ERP natif	Non	Non	Partiel	Non	Non	Enferme l'entreprise dans un seul ERP
Solution proposée	Oui	Oui	Oui	Oui	Oui	—

La colonne «versions ERP» mérite un mot. Un progiciel évolue, et ses versions majeures modifient la structure des données qu'il expose : une intégration construite sur une version peut cesser de fonctionner chez un client qui met la sienne à jour. Les plateformes étudiées échappent à cette difficulté en restant à la surface, à travers une API générique — d'où la mention «sans objet» ; un module natif la subit au contraire de plein fouet, puisqu'il doit être porté à chaque version. S'intégrer en profondeur tout en résistant aux montées de version est donc une exigence à part entière, dont le chapitre 5 détaille le traitement.

Ce constat oriente la conception du système autour de quatre partis pris : un adaptateur ERP commun plutôt qu'un développement par ERP, une intégration qui s'adapte à la version rencontrée chez le client plutôt que d'en supposer une seule, une isolation des données par entreprise cliente plutôt qu'un déploiement dédié à chacune, et une chaîne d'encaissement traitée comme un circuit de responsabilité — le livreur déclare la somme remise, un tiers la compte, un responsable arbitre les écarts — plutôt que comme un simple montant saisi à la livraison.

1.5 Méthodologie de développement

Le projet a été mené par un développeur unique, sur un périmètre fonctionnel qui s’est précisé au fil de l’avancement — notamment l’étendue exacte de l’intégration multi-ERP. Une méthode figée dès le départ aurait mal absorbé ces ajustements ; le tableau 1.3 compare les approches envisagées.

TAB. 1.3 : Comparaison des méthodologies de développement

Critère	Cycle en V	RUP	Scrum
Flexibilité	Faible	Moyenne	Forte
Adaptation au changement	Difficile	Partielle	Naturelle
Livraisons	Unique en fin	Par phases	Itératives (sprints)
Adapté à un projet solo	Non	Partiellement	Oui

Le choix s’est porté sur la méthode agile **Scrum**, pour ses cycles courts et sa capacité à réévaluer les priorités à chaque sprint plutôt qu’une seule fois en début de projet.

1.5.1 Présentation de Scrum

Scrum organise le développement en *sprints* : des cycles courts et de durée fixe, chacun livrant un incrément fonctionnel du produit. Le travail à faire est consigné dans un *Product Backlog*, dont un sous-ensemble est engagé pour chaque sprint dans un *Sprint Backlog*. Trois rôles structurent la méthode : le *Product Owner*, qui priorise le backlog ; le *Scrum Master*, qui fait respecter le processus ; l’équipe de développement, qui réalise les fonctionnalités.

1.5.2 Organisation du travail et outils

Le projet ayant été mené par un seul développeur, les trois rôles Scrum — Product Owner, Scrum Master et équipe de développement — ont été assumés par la même personne. Le suivi du Product Backlog et des Sprint Backlogs a été réalisé avec **GitLab Issues et Boards**, qui a servi de tableau kanban pour chaque sprint.

Le développement a été découpé en un sprint d’initialisation (sprint 0) suivi de cinq sprints fonctionnels, chacun clôturé par une revue et une rétrospective avant l’engagement du suivant. Le chapitre suivant détaille le sprint 0.

1.6 Conclusion

Ce chapitre a présenté le cadre général du projet : l'entreprise d'accueil ASM, la problématique de l'intégration ERP multiple sans dépendance ni interruption réseau, les objectifs et le périmètre retenus, ainsi que les limites des solutions existantes qui justifient cette approche. La méthodologie Scrum a été retenue pour sa capacité à absorber un périmètre encore mouvant. Le chapitre suivant présente le sprint 0 : l'analyse des besoins, la planification des sprints et l'étude technique préalable.

2.1 Introduction

Le chapitre précédent a posé le cadre : une entreprise de services numériques, un besoin de suivi de livraison adossé à des progiciels de gestion hétérogènes, et une méthode de travail itérative. Reste à transformer ce cadre en un plan de travail exécutable.

C'est l'objet du sprint 0. Contrairement aux cinq sprints qui le suivent, il ne livre aucune fonctionnalité : il produit les décisions sans lesquelles les autres ne pourraient pas commencer. Qui utilise le système et avec quels droits, ce qu'il doit savoir faire, dans quel ordre le construire, et avec quels outils.

Ce chapitre restitue ces quatre travaux dans cet ordre. Il se termine sur trois exigences qui, sans être des fonctionnalités, ont pesé plus lourd que toutes les autres sur les choix qui ont suivi.

2.2 Identification des acteurs

Un acteur est un rôle joué par une personne ou un système extérieur qui interagit avec la solution. Les acteurs retenus ne reprennent pas l'organigramme de l'entreprise cliente : ils suivent les *droits*. Deux postes qui ouvrent les mêmes écrans et font les mêmes opérations forment un seul acteur, même si leurs intitulés diffèrent. C'est pourquoi le système compte trois rôles internes et non quatre, comme le détaille le chapitre 4.

TAB. 2.1 : Acteurs du système et périmètre de chacun

Acteur	Description	Périmètre d'action
Administrateur	Responsable du paramétrage de l'installation pour son entreprise.	Comptes et rôles, correspondance des champs avec le progiciel, référentiels d'exploitation : chauffeurs, véhicules et zones. Hérite de toutes les actions du dispatcheur.
Dispatcheur	Il transforme au quotidien les commandes en tournées et arbitre les aléas.	Import des commandes, planification, affectation, validation, traitement des exceptions et des retours, comptage et arbitrage des remises de caisse, synchronisation des dépôts depuis le progiciel.
Responsable	Supervise l'activité sans intervenir dessus.	Consultation des livraisons, des tournées, des rapports, des espèces en circulation et du journal d'audit. Aucune action de modification.
Livreur	Exécute les tournées sur le terrain depuis l'application mobile.	Consultation de sa tournée du jour, changements de statut, preuve de livraison, déclaration d'échec, encaissement, transfert de garde, collecte des retours.
Destinataire	Client final du chargeur.	Consultation de l'avancement de son colis par un lien public, sans compte ni authentification, et demande de retour d'un article reçu.

Deux systèmes extérieurs complètent ce tableau sans être des acteurs au sens strict, puisqu'ils n'ont ni intention ni objectif propre : le **progiciel de gestion** de l'entreprise cliente — Odoo ou ERPNext —, source des commandes et destinataire des statuts, et le **serveur d'identité** Keycloak, qui authentifie les utilisateurs internes. Ils apparaissent dans le diagramme des cas d'utilisation sous forme de composants stéréotypés, à droite de la frontière du système.

2.3 Analyse et spécification des besoins

2.3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit savoir faire. Ils sont regroupés en huit domaines, qui serviront ensuite de découpage au *product backlog*.

Gestion des identités et des accès. Le système doit authentifier les utilisateurs internes, leur attribuer un rôle, et n'autoriser chaque opération que si le rôle porte la permission correspondante. Il doit également héberger plusieurs entreprises clientes sur une même installation, sans qu'aucune ne puisse atteindre les données d'une autre.

Intégration au progiciel de gestion. Le système doit importer les commandes prêtes à être livrées, puis renvoyer les statuts de livraison, les quantités réellement livrées et les preuves associées. Cette intégration doit fonctionner avec des progiciels différents, et avec des versions différentes d'un même progiciel, sans réécriture du cœur applicatif. L'administrateur doit pouvoir établir lui-même la correspondance entre les champs de son progiciel et ceux du système.

Référentiel logistique. Le système doit gérer les chauffeurs, les véhicules et les zones de couverture. Les dépôts, en revanche, ne sont pas saisis : ils reflètent les entrepôts du progiciel et sont récupérés par synchronisation, afin d'éviter un double référentiel qui divergerait au premier changement.

Planification et affectation. Le dispatcheur doit pouvoir construire une tournée à partir d'un ensemble de livraisons, y affecter un chauffeur et un véhicule, réordonner les arrêts, demander une optimisation automatique du parcours, puis valider la tournée pour la rendre visible au chauffeur.

Exécution terrain. Le chauffeur doit consulter sa tournée, déclarer son départ, marquer son arrivée à chaque arrêt, livrer ou déclarer un échec motivé, et collecter une preuve de livraison comportant une photographie du colis et une photographie du bon signé. Un transfert de garde entre deux chauffeurs doit être possible en cours de journée, avec confirmation du receveur.

Encaissement et remise de caisse. Lorsque la commande est payable à la livraison, le chauffeur doit enregistrer le montant encaissé et son mode de règlement. En fin de tournée, il déclare le total remis ; une autre personne compte la somme reçue et arbitre l'écart éventuel.

Suivi et traitement des aléas. Le dispatcheur doit voir l'avancement des livraisons en temps réel, être alerté des retards et des échecs, réaffecter une livraison bloquée, et enregistrer les retours de marchandise. Le destinataire doit pouvoir suivre son colis par un lien public.

Pilotage et exploitation. Le responsable doit disposer d'indicateurs — taux de réussite, motifs d'échec, respect des délais, performance par chauffeur et par zone — et pouvoir exporter des documents. L'administrateur doit disposer d'une console de santé qui expose l'état de l'intégration et permet de relancer une synchronisation en échec. Un assistant interne doit enfin permettre à chacun d'interroger l'activité de son entreprise en langage naturel, dans la limite de ce que son rôle l'autorise à consulter.

2.3.2 Besoins non fonctionnels

Les besoins non fonctionnels ne décrivent pas des fonctions mais des qualités attendues du système. Trois d'entre eux ont pesé plus lourd que les autres sur l'architecture et sont signalés comme tels.

TAB. 2.2 : Besoins non fonctionnels

Besoin	Exigence
Agnosticité ERP	Le cœur métier ne doit connaître aucun progiciel en particulier. Ajouter un progiciel, ou prendre en charge une nouvelle version, ne doit pas modifier le code métier. <i>Structurant.</i>
Isolation par client	Les données de chaque entreprise cliente doivent être séparées de façon structurelle, et non par un simple filtre applicatif qu'un oubli suffirait à contourner. <i>Structurant.</i>
Continuité hors ligne	L'application chauffeur doit rester utilisable sans réseau. Les actions réalisées hors couverture doivent être rejouées à la reconnexion sans produire de doublon. <i>Structurant.</i>
Sécurité	Communications chiffrées, jetons d'accès à durée limitée, contrôle des droits appliqué à la fois à la passerelle et dans chaque service.
Traçabilité	Toute opération sensible doit laisser une trace horodatée et attribuée à son auteur, consultable a posteriori.
Temps réel	Un changement d'état survenu sur le terrain doit se refléter sur le poste du dispatcheur sans rechargement manuel.
Performance	Les listes doivent être renvoyées par pages plutôt qu'en bloc, et tout critère de recherche exposé à l'utilisateur doit être indexé.
Portabilité	L'ensemble doit se déployer sur une machine unique par conteneurs, sans installation manuelle de dépendances.
Ergonomie	Interface d'administration en français, arabe et anglais ; application chauffeur conçue pour un usage à une main.

2.3.3 Diagramme global des cas d'utilisation

Le diagramme de la figure 2.1 synthétise les besoins fonctionnels sous forme de cas d'utilisation et les rattache à leurs acteurs. Il reste volontairement au premier niveau : le système en compte près de quatre-vingts une fois détaillés, les représenter tous rendrait la figure illisible. Le détail est porté par les diagrammes propres à chaque sprint, aux chapitres 4 à 8.

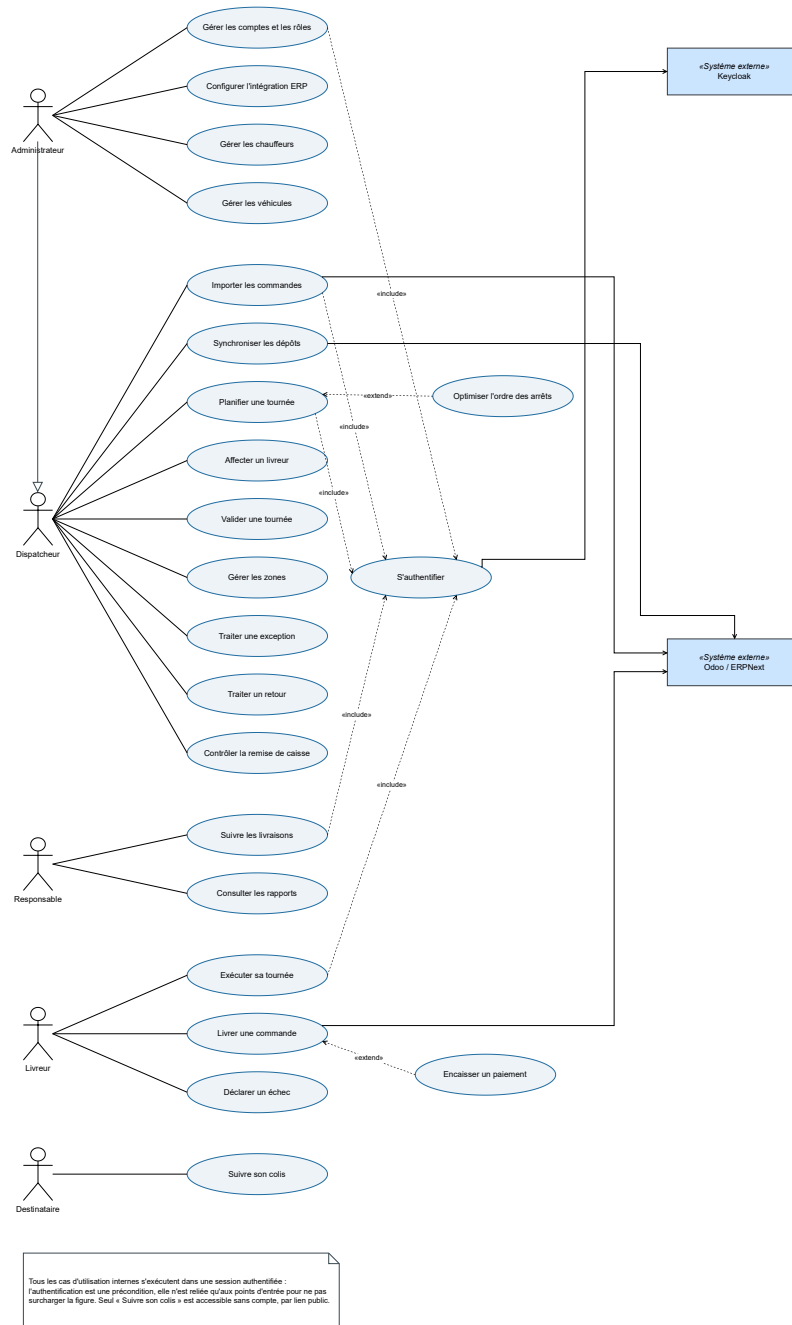


FIG. 2.1 : Diagramme global des cas d'utilisation

Trois éléments de lecture méritent d'être soulignés.

Une seule généralisation relie les acteurs : l'administrateur est un dispatcheur qui dispose en

plus du paramétrage, et ses permissions incluent effectivement toutes celles du dispatcheur. La flèche évite de redessiner une douzaine de traits.

Le responsable, lui, reste rattaché à ses propres cas. Il croise le dispatcheur sur les consultations sans jamais le contenir ni en dériver : il est seul à accéder au journal d’audit, et n’a aucune des permissions d’action. Superviser suppose de pouvoir relire ce que font ceux qui opèrent, sans pouvoir opérer soi-même ; c’est pourquoi les rôles sont déclarés à plat dans l’annuaire, chacun portant sa liste explicite, plutôt qu’en chaîne d’héritage. Le chapitre 4 détaille ce modèle.

Les liens « *include* » vers le cas *S’authentifier* ne sont tracés que depuis les points d’entrée. L’authentification est en réalité une précondition de tous les cas internes ; la relier à chacun surchargerait la figure sans rien apprendre au lecteur. Le destinataire, lui, en est véritablement exempt : son suivi passe par un lien public et n’exige aucun compte.

Les deux liens « *extend* » signalent des cas facultatifs. *Optimiser l’ordre des arrêts* étend *Planifier une tournée*, parce que le dispatcheur peut toujours ordonner ses arrêts lui-même. *Encaisser un paiement* étend *Livrer une commande*, parce que toutes les commandes ne sont pas payables à la livraison.

2.3.4 Diagramme de classes global

Le diagramme de la figure 2.2 présente les concepts manipulés par le système et leurs relations. Il fixe le vocabulaire employé dans tout le reste du rapport.

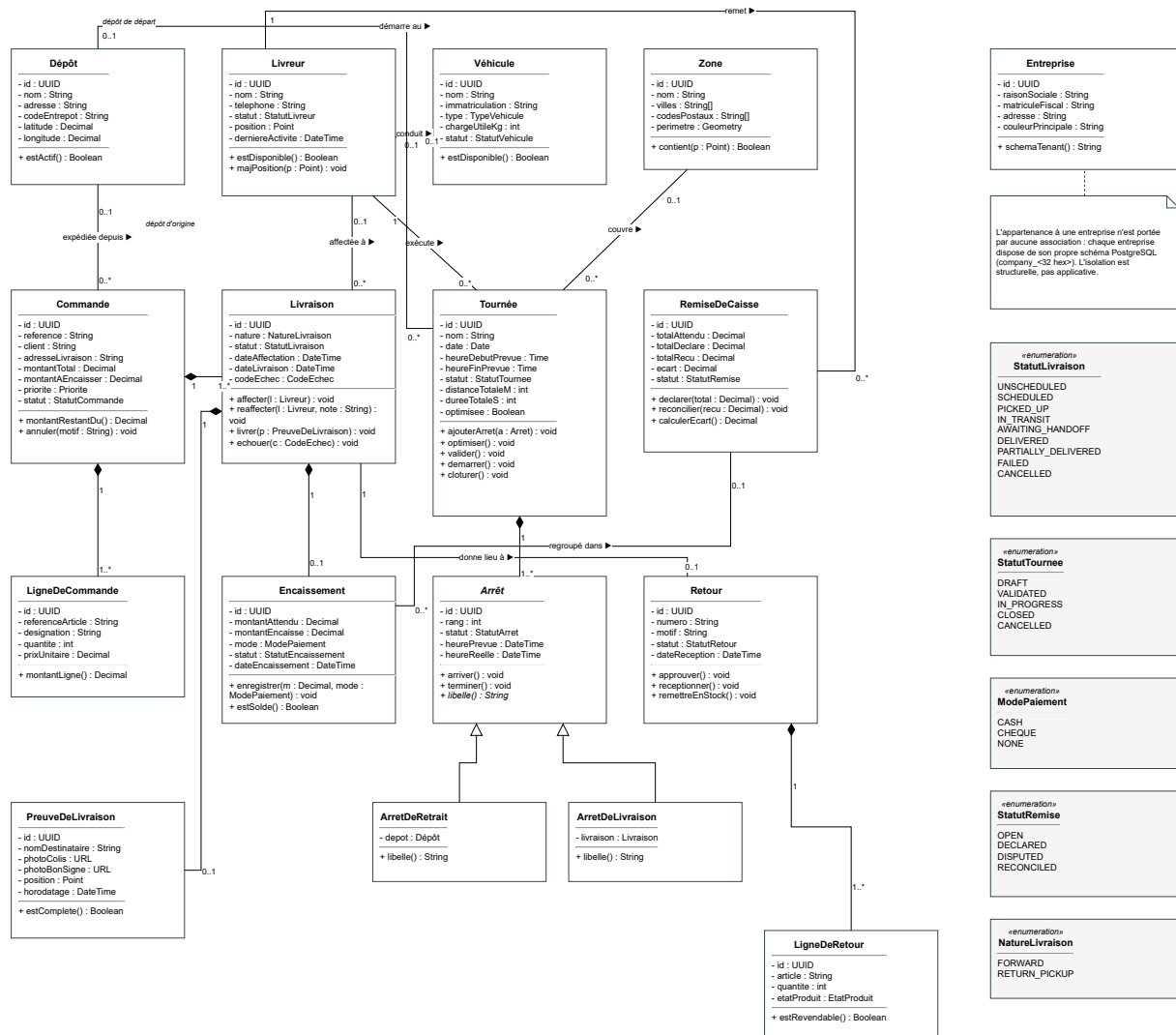


FIG. 2.2 : Diagramme de classes global

Il se lit de haut en bas, du référentiel vers l'exécution. La partie haute regroupe ce qui existe avant toute commande : dépôts, chauffeurs, véhicules et zones. Au centre, une **Commande** issue du progiciel se décompose en une ou plusieurs **Livraisons** ; le losange plein indique une composition, c'est-à-dire que les livraisons n'ont pas d'existence propre en dehors de leur commande. En parallèle, un chauffeur exécute des **Tournées**, elles-mêmes composées d'**Arrêts**.

La classe *Arrêt* est abstraite, et c'est le point le moins évident du modèle. Un arrêt de retrait désigne un dépôt où le chauffeur charge ; un arrêt de livraison désigne une livraison à remettre. Ce ne sont pas les mêmes objets, d'où la généralisation : les représenter par un simple attribut «type» aurait masqué une différence de structure réelle.

La partie basse porte les traces laissées par l'exécution : une **PreuveDeLivraison**, un **Encaissement** lorsque le client règle à la porte, et un **Retour** si la marchandise revient. Les encaissements d'un chauffeur sont ensuite regroupés dans une **RemiseDeCaisse**, ce qui

matérialise la séparation des responsabilités : celui qui déclare la somme n'est pas celui qui la valide.

Enfin, la classe **Entreprise** n'est reliée à rien, et c'est délibéré. L'appartenance d'une donnée à une entreprise cliente n'est pas portée par une clé étrangère mais par un schéma de base de données distinct. Le chapitre 3 détaille ce mécanisme.

2.4 Gestion du projet avec Scrum

2.4.1 Product Backlog

Le *product backlog* traduit les besoins fonctionnels en éléments réalisables et priorisés. Ils ont été formulés sous forme de *user stories*, selon le gabarit « en tant que *rôle*, je veux *action*, afin de *bénéfice* », puis regroupés en huit épopées correspondant aux domaines de la section précédente.

Deux échelles complètent chaque élément. La **priorité** suit la méthode MoSCoW — *Must have* pour l'indispensable, *Should have* pour l'important non bloquant, *Could have* pour le souhaitable. L'**estimation** est exprimée en points de complexité selon la suite de Fibonacci ; il s'agit d'une complexité relative entre éléments, non d'une durée. Trois repères ont servi d'étalon tout au long de l'estimation : 3 points pour une gestion de liste sans règle particulière, 8 points pour une fonction qui traverse plusieurs composants, 21 points pour une fonction qui impose une décision d'architecture.

Les *Must* pèsent 240 points, soit les deux tiers du backlog. C'est au-dessus de la proportion habituellement conseillée, et la raison est assumée : l'isolation des données, l'indépendance vis-à-vis du progiciel et la continuité hors ligne n'admettent pas de version dégradée. La marge de manœuvre est donc portée par les 39 points de *Could*, qui pouvaient être abandonnés sans dénaturer le produit.

TAB. 2.3 : Product Backlog du projet

Réf.	User story	Priorité	Est.	Sprint
Épopée 1 — Identités, autorisations et multi-tenance				
US-01	En tant qu'utilisateur, je veux m'authentifier par un compte unique afin d'accéder aux écrans correspondant à mon rôle.	Must	8	1
US-02	En tant qu'administrateur, je veux créer des comptes et leur attribuer un rôle afin de contrôler qui accède à quoi.	Must	3	1

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-03	En tant qu’administrateur, je veux que chaque opération vérifie la permission portée par le rôle de l’utilisateur afin que personne ne puisse dépasser ses droits.	Must	8	1
US-04	En tant qu’entreprise cliente, je veux que mes données soient inaccessibles aux autres entreprises hébergées afin de garantir ma confidentialité.	Must	13	1
Épopée 2 — Intégration au progiciel de gestion				
US-05	En tant que dispatcheur, je veux importer les commandes prêtes depuis le progiciel afin de ne pas les ressaisir.	Must	13	2
US-06	En tant qu’administrateur, je veux relier les champs de mon progiciel à ceux du système afin d’adapter l’intégration à mon paramétrage.	Must	13	2
US-07	En tant qu’entreprise, je veux pouvoir brancher la solution sur n’importe quel progiciel de gestion afin de ne dépendre d’aucun éditeur.	Must	21	2
US-08	En tant que dispatcheur, je veux que le statut d’une livraison remonte automatiquement dans le progiciel afin que la facturation suive.	Must	8	2
US-09	En tant que dispatcheur, je veux qu’une synchronisation interrompue soit réessayée puis signalée afin qu’aucune information ne se perde silencieusement.	Must	13	2
US-10	En tant que dispatcheur, je veux récupérer les entrepôts du progiciel comme dépôts afin d’éviter un double référentiel.	Should	5	2
US-11	En tant qu’administrateur, je veux que le système fonctionne avec plusieurs versions du même progiciel afin de ne pas imposer de montée de version au client.	Must	13	2
Épopée 3 — Référentiel logistique				
US-12	En tant qu’administrateur, je veux gérer la liste des chauffeurs afin de refléter l’équipe réelle.	Must	3	1

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-13	En tant qu'administrateur, je veux gérer le parc de véhicules avec leur capacité afin de pouvoir les affecter.	Should	3	1
US-14	En tant qu'administrateur, je veux définir des zones géographiques afin de regrouper les livraisons par secteur.	Should	8	1
Épopée 4 — Planification et affectation				
US-15	En tant que dispatcheur, je veux construire une tournée à partir de livraisons afin d'organiser la journée.	Must	13	3
US-16	En tant que dispatcheur, je veux affecter un chauffeur et un véhicule à une tournée afin d'en désigner le responsable.	Must	8	3
US-17	En tant que dispatcheur, je veux obtenir un ordre de passage optimisé afin de réduire la distance parcourue.	Should	13	3
US-18	En tant que dispatcheur, je veux valider une tournée afin de la rendre visible au chauffeur.	Must	3	3
US-19	En tant que dispatcheur, je veux connaître l'heure d'arrivée estimée à chaque arrêt afin de détecter les retards.	Could	8	3
Épopée 5 — Exécution terrain				
US-20	En tant que chauffeur, je veux consulter ma tournée du jour sur mon téléphone afin de savoir où aller.	Must	8	4
US-21	En tant que chauffeur, je veux déclarer une livraison réussie ou échouée avec un motif afin que le bureau soit informé.	Must	8	4
US-22	En tant que chauffeur, je veux enregistrer une preuve de livraison avec photographies afin de couvrir l'entreprise en cas de litige.	Must	13	4
US-23	En tant que chauffeur, je veux continuer à travailler sans réseau et voir mes actions transmises au retour de la couverture afin de ne jamais être bloqué.	Must	21	4

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-24	En tant que chauffeur, je veux transmettre un colis à un collègue avec confirmation afin que la responsabilité soit tracée.	Could	13	4
Épopée 6 — Encaissement et remise de caisse				
US-25	En tant que chauffeur, je veux enregistrer le montant encaissé et son mode de règlement afin d'en garder la trace.	Must	8	4
US-26	En tant que chauffeur, je veux déclarer le total remis en fin de tournée afin de clore ma journée.	Must	3	4
US-27	En tant que dispatcheur, je veux compter la somme reçue et arbitrer l'écart afin que la caisse soit vérifiable.	Must	8	4
Épopée 7 — Suivi, aléas et retours				
US-28	En tant que dispatcheur, je veux voir l'avancement des livraisons sans recharger la page afin de réagir immédiatement.	Must	13	3
US-29	En tant que dispatcheur, je veux être alerté des retards et des échecs afin de traiter les exceptions en priorité.	Should	5	3
US-30	En tant que dispatcheur, je veux réaffecter une livraison bloquée à un autre chauffeur afin de sauver la journée.	Must	5	3
US-31	En tant que destinataire, je veux suivre mon colis par un lien reçu afin de connaître son avancement sans créer de compte.	Should	8	3
US-32	En tant que destinataire, je veux demander le retour d'un article reçu afin que ma demande soit enregistrée et suivie.	Should	3	3
US-33	En tant que dispatcheur, je veux approuver ou refuser une demande de retour et déclencher la reprise en stock afin de garder la main sur ce qui revient.	Should	8	3
US-34	En tant que chauffeur, je veux collecter un retour chez le destinataire afin de le ramener au dépôt.	Should	5	4
Épopée 8 — Pilotage et exploitation				

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-35	En tant que responsable, je veux consulter les indicateurs de performance afin de piloter l'activité.	Should	13	5
US-36	En tant que responsable, je veux exporter un rapport de tournée afin de le transmettre ou de l'archiver.	Could	5	5
US-37	En tant qu'administrateur, je veux une console indiquant l'état de l'intégration afin de distinguer une panne du progiciel d'une panne du système.	Must	13	5
US-38	En tant qu'administrateur, je veux relancer une synchronisation en échec afin de réparer sans intervention technique.	Should	5	5
US-39	En tant qu'administrateur, je veux savoir qui a fait quoi et quand sur les données sensibles afin de pouvoir répondre à un audit.	Should	8	5
US-40	En tant qu'utilisateur, je veux poser une question en langage naturel sur l'activité de mon entreprise afin d'obtenir la réponse sans parcourir plusieurs écrans.	Could	13	5

Le backlog totalise quarante éléments pour 363 points de complexité. Il a été tenu dans **GitLab Issues**, chaque épopée correspondant à une étiquette et chaque sprint à un tableau kanban dédié.

2.4.2 Planification des sprints

Le projet s'est déroulé du 2 mars au 17 août 2026, soit vingt-quatre semaines. Il a été découpé en un sprint d'initialisation et cinq sprints fonctionnels de trois à cinq semaines, chacun clos par une revue et une rétrospective avant l'engagement du suivant.

L'ordre des sprints obéit à une contrainte simple : chaque sprint ne doit dépendre que de ce qui a déjà été livré. L'isolation par entreprise et le contrôle des droits viennent en premier, parce que toute donnée créée ensuite doit déjà être rattachée à un client et protégée — les rajouter après coup aurait imposé de reprendre l'ensemble du code écrit entre-temps. L'intégration au progiciel vient juste après, parce qu'elle alimente le système en commandes : sans elle, les sprints suivants auraient travaillé sur des données saisies à la main, donc sur un cas qui n'existe pas en production.

La priorité MoSCoW n'intervient donc pas dans cet ordre : c'est la dépendance technique qui le fixe. Elle joue à l'intérieur de chaque sprint, où elle désigne ce qui saute en premier si la fin du cycle approche. C'est ce qui explique qu'un *Could* soit livré dès le sprint 3 alors que des *Must* attendent le sprint 5.

TAB. 2.4 : Planification des sprints

Sprint	Objectif	Période	Durée	Points
Sprint 0	Cadrage, besoins et étude technique	2 — 20 mars	3 sem.	—
Sprint 1	Identité, autorisation et référentiels	23 mars — 17 avril	4 sem.	46
Sprint 2	Intégration ERP agnostique	20 avril — 22 mai	5 sem.	86
Sprint 3	Planification des tournées et suivi temps réel	25 mai — 19 juin	4 sem.	87
Sprint 4	Application chauffeur et hors ligne	22 juin — 17 juillet	4 sem.	87
Sprint 5	Exploitation, résilience et assistance	20 juillet — 7 août	3 sem.	57
Recette, rédaction et soutenance		10 — 17 août	1 sem.	—

Les sprints 2, 3 et 4 concentrent les deux tiers des points. Le premier parce que l'indépendance vis-à-vis du progiciel exige une couche d'abstraction complète avant le moindre gain visible ; les deux suivants parce qu'ils portent le cœur métier, côté bureau puis côté terrain.

Rapportée à la durée, la charge passe de 11 points par semaine au sprint 1 à près de 22 aux sprints 3 et 4, avant de redescendre. Cette progression n'est pas un hasard de découpage : le premier sprint a été ralenti par la mise en place de l'environnement et de la chaîne de déploiement, dont les suivants ont hérité sans en repayer le coût.

2.5 Étude technique

2.5.1 Environnement matériel

Le développement a été mené sur un poste unique, qui a également hébergé l'ensemble des services conteneurisés — bases de données, courtier de messages, serveur d'identité, moteur de calcul d'itinéraires et instances de progiciel de test. Faire tourner tout cela ensemble est ce qui a imposé les 24 Go de mémoire.

TAB. 2.5 : Environnement matériel de développement

Élément	Caractéristique
Processeur	Intel Core i5-12450H, 12 ^e génération
Mémoire vive	24 Go
Système d’exploitation	Windows 11 Professionnel
Exécution des services	Docker Desktop, moteur WSL 2
Essais mobiles	Appareil Android physique et émulateur

2.5.2 Environnement logiciel de développement

TAB. 2.6 : Outils de développement et de collaboration

Outil	Usage dans le projet
Visual Studio Code	Éditeur principal pour le code Java, TypeScript et Dart
Android Studio	Compilation, émulation et débogage de l’application chauffeur
Git et GitLab	Gestion de versions, suivi du backlog par issues et tableaux, intégration continue
Docker Desktop	Exécution locale de l’ensemble des services et de leurs dépendances
Postman	Essais manuels des interfaces de programmation
DBeaver	Inspection des schémas et des données PostgreSQL
draw.io	Conception des diagrammes UML du présent rapport
L ^A T _E X (XeLaTeX)	Rédaction du rapport

2.5.3 Technologies retenues

Le choix des technologies a été arrêté au terme du sprint 0, chaque brique étant retenue pour une raison rattachable à un besoin de la section 2.3. Le chapitre 3 en détaille la mise en œuvre.

TAB. 2.7 : Technologies retenues et motif du choix

Couche	Technologie	Motif du choix
Services	Spring Boot 3.3.5, Java 17	Écosystème mature pour les architectures en services, avec prise en charge native de la sécurité par jetons et du multi-schéma
Passerelle	Spring Cloud Gateway	Point d'entrée unique : validation des jetons et application des droits avant tout accès aux services
Interface web	React 19, Vite, TypeScript	Rendu réactif adapté à un tableau de dispatch qui se met à jour en continu
Application mobile	Flutter	Base de code unique, et stockage local performant sur lequel repose le mode hors ligne
Base de données	PostgreSQL 16	Schéma par entreprise cliente pour l'isolation, type jsonb pour les données de forme variable, absence de coût de licence
Messagerie	RabbitMQ 4	Découplage des échanges avec le progiciel, relances et file de rebut intégrées
Identité	Keycloak	Serveur OpenID Connect éprouvé, gestion native des organisations
Fichiers	MinIO	Stockage compatible S3 des photographies de preuve, déployable localement
Itinéraires	OSRM	Moteur de calcul auto-hébergé, sans coût par requête ni dépendance à un service externe

Sur le choix de la base de données. PostgreSQL a été retenu pour deux raisons directement liées aux besoins structurants. La première est l'isolation par entreprise cliente : PostgreSQL permet de créer un schéma par client au sein d'une même base et d'en changer dynamiquement à chaque requête, mécanisme sur lequel repose toute la multi-tenance décrite au chapitre 3. La seconde est le stockage semi-structuré : le détail des lignes de commande et les champs supplémentaires propres à chaque client varient d'un progiciel à l'autre, ce que le type jsonb absorbe sans multiplier les tables. S'y ajoute l'absence de coût de licence, cohérente avec une solution destinée à être déployée chez plusieurs clients.

2.6 Conclusion

Ce sprint d'initialisation a produit quatre livrables : la liste des acteurs et de leurs droits, la spécification des besoins fonctionnels et non fonctionnels, les deux diagrammes qui fixent le vocabulaire du système, et un backlog de quarante éléments réparti sur cinq sprints.

Son apport principal n'est pourtant pas cette production documentaire. Il tient à trois exigences

que l'analyse a fait remonter au rang de contraintes structurantes : l'ouverture à plusieurs progiciels, l'hébergement de plusieurs entreprises clientes sur une même installation, et la continuité de service en l'absence de réseau. Aucune n'est une fonctionnalité visible pour l'utilisateur, et pourtant toutes trois ont déterminé l'architecture, l'ordre des sprints et une partie des choix technologiques.

Le chapitre suivant présente cette architecture. Les cinq chapitres qui lui succèdent déroulent ensuite les sprints dans l'ordre du tableau 2.4.

3.1 Introduction

Le sprint 0 a dit ce que le système doit faire et pour qui. Il reste à décider comment il est construit. C'est l'objet de ce chapitre, placé avant les sprints parce que les décisions qu'il contient ont été prises une fois, au début, et n'ont plus été remises en cause ensuite : chaque sprint s'est inséré dans cette structure au lieu de la redéfinir.

Quatre questions sont traitées dans l'ordre où elles se sont posées. Faut-il une application unique ou plusieurs services ? Où vivent les données une fois le système découpé ? Comment les données d'une entreprise cliente restent-elles hors de portée des autres ? Et comment les services se transmettent-ils l'information ?

3.2 Une application unique ou plusieurs services

Le premier choix engage tous les autres. Une application unique — un *monolithe* — regroupe tout le code dans un seul programme, avec une seule base de données. Une architecture en services découpe le programme en plusieurs applications autonomes, chacune avec sa base, qui communiquent par le réseau.

Le tableau 3.1 résume ce que chaque approche apporte et ce qu'elle coûte.

TAB. 3.1 : Application unique ou services séparés

Critère	Application unique	Services séparés
Mise en route	Immédiate	Plus longue : réseau, messagerie, configuration
Déploiement	Tout ou rien	Un service à la fois
Panne	Tout s'arrête	Le reste continue de fonctionner
Données	Une base, jointures libres	Une base par service, pas de jointure entre services
Montée en charge	On duplique l'ensemble	On duplique la partie sollicitée
Complexité	Concentrée dans le code	Déplacée vers les échanges entre services

Les services séparés ont été retenus pour trois raisons propres à ce projet.

La première est l'intégration au progiciel du client. Le code qui parle à Odoo ou à ERPNext est le plus instable du système : il change à chaque nouveau client et à chaque montée de version. L'isoler dans un service à part permet de le modifier sans redéployer le cœur métier, et de le laisser tomber en panne sans arrêter les livraisons du jour.

La deuxième est l'application mobile. Les chauffeurs envoient des positions et des changements de statut à un rythme sans commune mesure avec le trafic du bureau. Séparer le service qui les reçoit évite qu'une journée chargée sur le terrain ne ralentisse l'écran du dispatcheur.

La troisième est la possibilité de remplacer une partie sans toucher au reste. Un service ne montre à ses voisins que son interface : ce qu'il fait à l'intérieur, la base qu'il utilise, la bibliothèque sur laquelle il s'appuie ne regardent que lui. On peut donc le réécrire, changer sa base ou son moteur de calcul sans que les autres s'en aperçoivent. Dans une application unique, la même opération suppose de démêler des dépendances qui se sont installées un peu partout dans le code, faute d'une frontière qui les empêche de le faire.

Cette frontière vaut aussi vers l'extérieur. Ce qui se trouve derrière l'interface d'un service n'a pas à être du code écrit ici. Le service Chauffeur, par exemple, expose des chauffeurs, leur disponibilité et leur position ; le reste du système n'en sait rien de plus, et n'a pas à savoir d'où vient l'information.

Une entreprise qui préférerait confier ses tournées à un transporteur plutôt qu'à ses propres chauffeurs n'aurait donc qu'à mettre, à cette place, un service qui parle à l'interface de ce transporteur. Le service de livraison, la passerelle et le back-office continueraient de fonctionner sans une ligne de changement. Le système est prêt à s'appuyer sur des prestataires extérieurs sans être réécrit pour autant.

Le découpage suit les **domaines métier** identifiés au chapitre précédent, et non les couches techniques. Il n'existe pas de « service base de données » ni de « service interface » : chaque service porte un domaine complet, de son interface HTTP jusqu'à ses tables. Un développeur qui doit corriger le calcul d'une tournée sait donc où aller, sans traverser trois applications.

Ce choix a un prix, et il vaut mieux le nommer que le laisser découvrir au lecteur. Il tient en trois points.

Le premier est l'**incertitude sur le résultat d'un appel**. Un appel de méthode dans une application unique peut évidemment échouer ; mais on sait toujours qu'il a échoué, et la transaction en cours annule tout d'un bloc. Un appel entre deux services peut, lui, expirer sans réponse : l'appelant ne sait pas si le travail a été fait ou non. C'est ce doute, et non la panne, qui coûte cher : réessayer risque de faire l'opération deux fois, ne pas réessayer risque de ne

pas la faire du tout. Il faut donc rendre les opérations rejouables sans double effet, ce que le chapitre 5 détaille.

Le deuxième est l'**absence de transaction commune**. Deux écritures dans deux services ne peuvent pas être annulées ensemble. Il faut soit accepter un décalage passager entre les deux, soit prévoir une opération de rattrapage.

Le troisième est le **coût d'exploitation** : plus de conteneurs à démarrer, plus de journaux à consulter, et une panne plus longue à localiser puisqu'elle peut venir de n'importe quel maillon de la chaîne.

Trois raisons d'un côté, trois coûts de l'autre : la balance ne se lit pas au nombre. Les coûts sont techniques, ils se paient une fois et le même mécanisme sert partout ensuite. Les raisons, elles, reviennent à chaque évolution du système : un client qui met son progiciel à jour, une journée chargée sur le terrain, un composant à remplacer. C'est ce qui a fait pencher la décision.

Les sections 3.4 et 3.6 montrent comment ces trois points sont traités plutôt qu'ignorés.

3.3 Vue d'ensemble

La figure 3.1 présente l'ensemble du système : les applications qui l'utilisent à gauche, les services au centre, les composants d'infrastructure autour.

Architecture microservices d'ASM Track - Vue d'ensemble

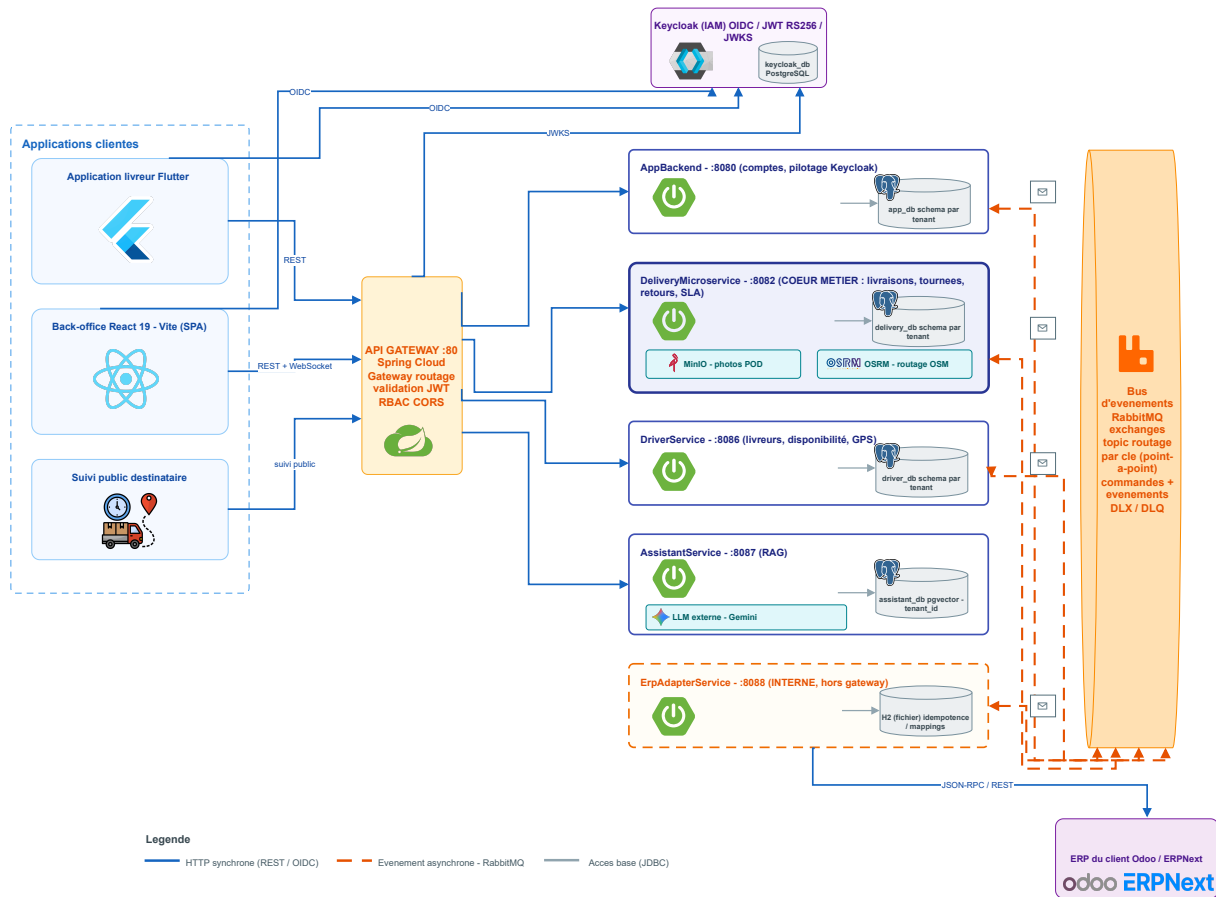


FIG. 3.1 : Vue d'ensemble de l'architecture

Trois trajets se lisent sur cette figure.

Le **trajet ordinaire** part d'une application cliente, traverse la passerelle et atteint un service, qui répond en interrogeant sa base. C'est le chemin de presque toutes les requêtes.

Le **trajet asynchrone**, en pointillés, ne passe par la passerelle ni par personne d'autre : un service dépose un message sur le bus et poursuit son travail sans attendre. Le destinataire le traitera quand il le pourra.

Le **trajet vers le progiciel du client** est le seul à sortir du système. Il est entièrement confié à l'adaptateur, qui est aussi le seul service à ne pas être exposé par la passerelle : aucune application, aucun navigateur ne peut l'appeler directement.

3.3.1 La passerelle

Toutes les requêtes venues de l'extérieur entrent par la passerelle, qui remplit quatre rôles.

Elle **achemine** la requête vers le bon service d'après le début de son adresse : quarante-cinq règles couvrent l'ensemble des routes exposées.

Elle **vérifie le jeton** d'authentification auprès du serveur d'identité, et refuse la requête si la signature ou la date de validité ne conviennent pas.

Elle **contrôle les droits** avant de transmettre. Si le rôle de l'utilisateur ne porte pas la permission attendue, la requête n'atteint jamais le service. Ce contrôle est ensuite refait à l'intérieur de chaque service, pour la raison exposée au chapitre 4.

Elle **réécrit les en-têtes d'identité**. Ceux qui arrivent du client sont effacés, puis reconstruits à partir du contenu du jeton. Un appelant ne peut donc pas se déclarer administrateur en ajoutant un en-tête à sa requête.

3.3.2 Les services

Le tableau 3.2 récapitule les six services, ce dont chacun est responsable et où il range ses données.

TAB. 3.2 : Les services et leurs responsabilités

Service	Responsabilité	Données
Passerelle	Acheminement, vérification du jeton, droits, en-têtes d'identité	Aucune
Comptes	Utilisateurs, rôles, entreprises clientes, pilotage du serveur d'identité	Base des comptes
Livraison	Cœur métier : commandes, livraisons, tournées, preuves, encaissements, retours	Base des livraisons ; photographies dans le stockage d'objets
Chauffeur	Fiches des chauffeurs, disponibilité, positions	Base des chauffeurs
Assistant	Questions posées en langage naturel sur l'activité de l'entreprise	Base avec index vectoriel
Adaptateur ERP	Seul point de contact avec le progiciel du client	Base locale des correspondances

Le service de livraison est nettement plus gros que les autres. Ce n'est pas un défaut de découpage mais le reflet du métier : la commande, la livraison, la tournée et la preuve forment un tout dont les parties se modifient ensemble. Les séparer aurait imposé des transactions réparties sur plusieurs services, c'est-à-dire remplacé une difficulté simple par une difficulté beaucoup plus lourde.

3.3.3 Les composants d'infrastructure

Quatre composants ne portent aucune règle métier mais rendent un service à tous les autres.

Le **serveur d'identité** détient les comptes et délivre les jetons. Aucun service ne conserve de mot de passe.

Le **bus de messages** transporte les échanges asynchrones et conserve les messages tant qu'ils n'ont pas été traités.

Le **stockage d'objets** reçoit les photographies des preuves de livraison. Elles ne sont pas mises en base : un fichier de plusieurs centaines de kilo-octets n'a rien à faire dans une table.

Le **moteur d'itinéraires** calcule distances et durées entre deux points à partir de données cartographiques libres. Il est hébergé avec le reste plutôt qu'appelé chez un fournisseur, ce qui évite de payer à la requête et de dépendre d'un service extérieur.

3.4 Du modèle de classes aux bases de données

Le chapitre 2 a présenté un modèle de classes unique, où toutes les entités apparaissent ensemble. C'est le modèle du métier, et il reste valable. Mais le découpage en services lui impose une contrainte que ce diagramme ne montre pas : ces entités ne vivent pas dans la même base.

La figure 3.2 montre cette répartition.

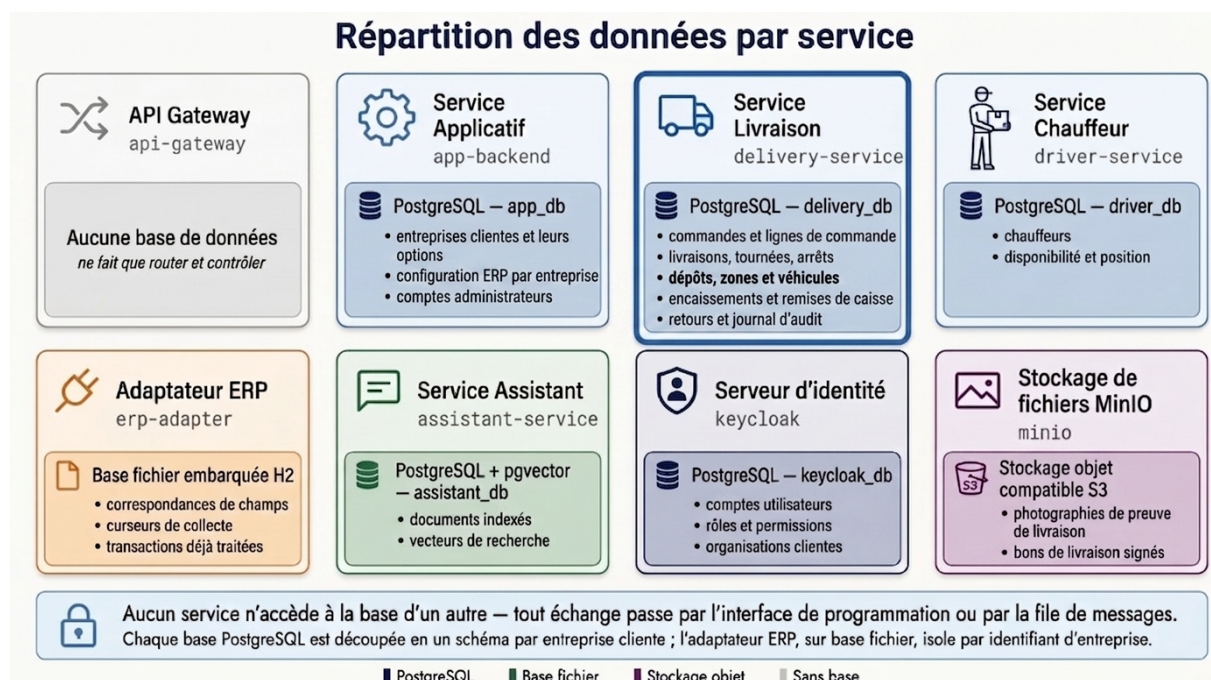


FIG. 3.2 : Répartition des données entre les services

La règle est simple et sans exception : **un service ne lit que sa propre base**. Aucun ne va chercher une donnée directement dans la base d'un autre, même en lecture seule.

Ce raccourci est pourtant tentant, parce qu'il marche tout de suite. Le problème vient après : dès qu'il existe, plus personne ne peut modifier ces tables sans risquer de casser un service qu'il n'avait pas vu.

3.4.1 Ce que devient une relation entre deux services

Cette règle a une conséquence directe sur le modèle. Une livraison est affectée à un chauffeur, mais la livraison est rangée dans la base des livraisons et le chauffeur dans celle des chauffeurs. La relation existe toujours dans le métier ; elle ne peut plus être une clé étrangère.

Elle devient un **simple identifiant** : la livraison conserve l'identifiant du chauffeur, sans que la base ne garantisse quoi que ce soit à son sujet. Deux différences en découlent, qu'il faut accepter en connaissance de cause.

La base ne vérifie plus rien. Dans une application unique, une clé étrangère interdit d'affecter une livraison à un chauffeur qui n'existe pas : la base refuse l'écriture. Ici, rien ne s'y oppose ; c'est au service de le vérifier lui-même, en interrogeant son voisin ou en réagissant au message qui lui apprend qu'un chauffeur a été supprimé.

La jointure disparaît. Dans une application unique, une seule requête suffit pour afficher une tournée avec le nom de son chauffeur. Ici il en faut deux : le service lit la tournée dans sa base, puis appelle le service Chauffeur pour obtenir le nom.

3.4.2 L'évolution du schéma

Les tables ne sont pas créées à la main. Chaque modification du schéma est écrite dans un fichier numéroté, appliqué automatiquement au démarrage du service et jamais modifié une fois livré : une correction donne lieu à un nouveau fichier. Le projet en compte cent cinquante-neuf, dont la grande majorité pour le service de livraison.

L'intérêt ne se limite pas à la simplicité. Chaque environnement reconstruit sa base de données à partir des mêmes fichiers de migration. Ainsi, les bases de test et d'exécution conservent la même structure et ne risquent pas de diverger. Lorsqu'un problème apparaît dans un environnement, il est donc possible de reproduire les mêmes conditions dans un autre afin de l'identifier et de le corriger plus facilement.

3.5 L'isolation des entreprises clientes

Une même installation héberge plusieurs entreprises clientes. Chacune doit voir ses données, et uniquement les siennes. C'est l'exigence la plus contraignante du projet : une seule fuite, une seule fois, suffirait à disqualifier la solution.

3.5.1 Trois façons de séparer, une seule retenue

La première consiste à ajouter une colonne «entreprise» à chaque table et à filtrer dessus. Elle est simple, et c'est sa faiblesse : la séparation ne tient qu'à la présence d'une condition dans chaque requête. Une seule requête écrite sans elle, et une entreprise voit les données d'une autre. Rien, dans la base, ne s'y oppose.

La deuxième consiste à donner une base complète à chaque entreprise. La séparation devient matérielle, donc incontestable, mais le coût suit le nombre de clients : autant de bases à sauvegarder, à migrer et à surveiller, et autant de connexions ouvertes en permanence.

La troisième, retenue ici, est le **schéma par entreprise**. Une base unique contient autant de schémas que d'entreprises, chacun avec son jeu complet de tables. Une requête envoyée dans un schéma ne peut pas atteindre les tables d'un autre, sans qu'aucune condition n'ait à être écrite. Les sauvegardes et les migrations, elles, restent celles d'une base unique.

Le nom du schéma est déduit de l'identifiant de l'entreprise : le préfixe `company_` suivi des trente-deux caractères de cet identifiant. Cette déduction est faite à un seul endroit du code, à partir d'un identifiant déjà reconnu comme tel, ce qui écarte la possibilité d'y glisser un nom de schéma arbitraire.

3.5.2 Le trajet d'une requête

Encore faut-il que le bon schéma soit choisi à chaque requête, sans que le développeur ait à y penser. La figure 3.3 détaille les huit étapes de ce trajet.

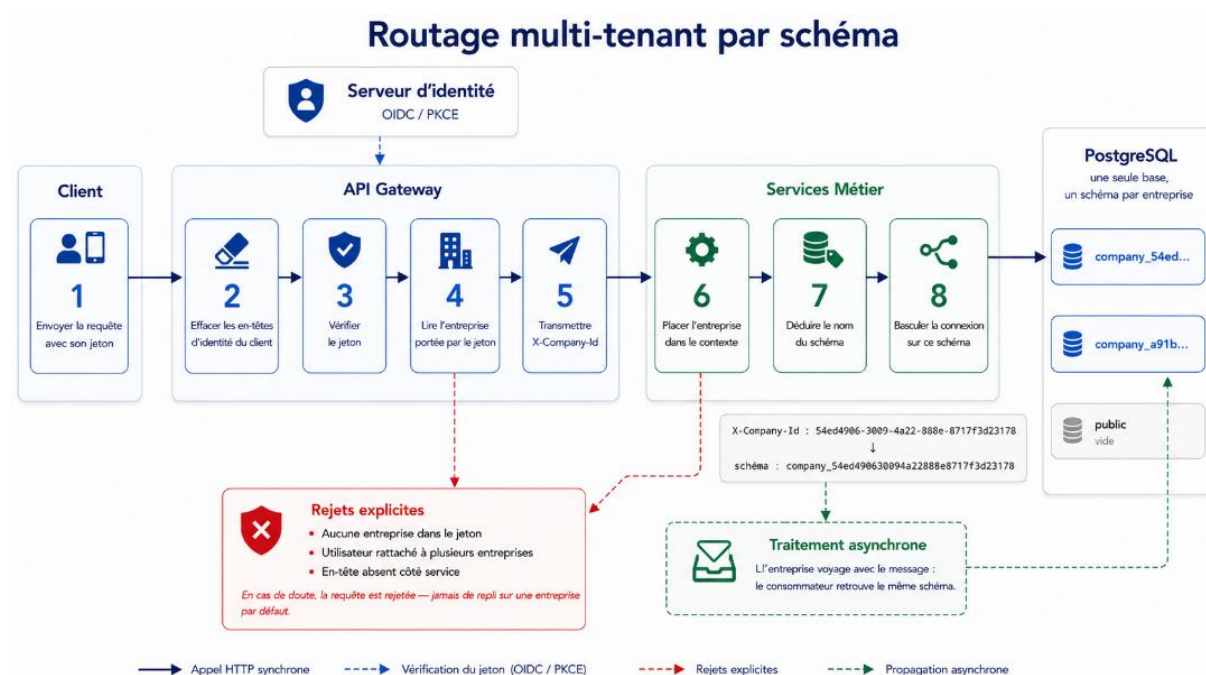


FIG. 3.3 : Le choix du schéma, de la requête jusqu'à la base

Les quatre premières étapes se déroulent dans la passerelle. Elle efface les en-têtes d'identité présents dans la requête, vérifie le jeton, y lit l'entreprise à laquelle l'utilisateur appartient, puis ajoute cette information à la requête qu'elle transmet. L'entreprise vient donc du jeton signé par le serveur d'identité, jamais de ce que l'appelant a bien voulu déclarer.

Les quatre suivantes se déroulent dans le service. Un filtre lit l'entreprise transmise et la place dans le contexte de la requête. Au moment d'ouvrir la connexion à la base, ce contexte est traduit en nom de schéma et la connexion bascule sur ce schéma. Le code métier, lui, écrit ses requêtes sans jamais mentionner l'entreprise ni le schéma : il ne peut donc pas oublier de le faire.

Le basculement porte sur la connexion que la transaction va réellement utiliser, et il est annulé avant que cette connexion ne retourne au réservoir partagé. Sans cette remise à zéro, la requête suivante hériterait du schéma de la précédente.

3.5.3 Le refus plutôt que le repli

Une règle traverse tout ce mécanisme : en cas de doute, la requête est rejetée. Jamais de repli sur une entreprise par défaut, jamais de valeur devinée. Quatre situations conduisent à un refus : le jeton ne porte aucune entreprise, l'utilisateur est rattaché à plusieurs entreprises, l'identifiant n'a pas la forme attendue, ou l'en-tête manque à l'entrée du service.

Le deuxième cas mérite une explication. Le problème n'est pas qu'un utilisateur travaille pour deux entreprises ; c'est que sa requête ne dit pas laquelle des deux elle concerne. Le système devrait alors deviner, et il devinerait mal : l'utilisateur verrait tantôt les données de l'une, tantôt celles de l'autre, sans qu'aucune erreur ne soit signalée. La règle est donc stricte : un utilisateur appartient à une entreprise et à une seule.

Prendre en charge le cas contraire ne demanderait d'ailleurs pas de revoir ce mécanisme, mais de faire choisir son entreprise à l'utilisateur au moment de la connexion, afin que le jeton n'en porte qu'une. C'est le choix qui manque aujourd'hui, non la capacité à l'appliquer.

Le quatrième cas garantit que la protection tient même si la passerelle est contournée. Un service appelé directement, sans en-tête d'entreprise, refuse la requête au lieu de servir des données.

3.5.4 Le cas des traitements asynchrones

Lorsqu'un message est traité ultérieurement par un autre service, la requête HTTP et le jeton d'authentification d'origine ne sont plus disponibles. Pour conserver le contexte de l'entreprise, son identifiant est donc ajouté dans l'en-tête du message au moment de sa publication. À sa réception, le service consommateur lit cet identifiant avant tout traitement et

détermine ainsi le schéma correspondant à l'entreprise concernée. L'opération est alors exécutée dans le même contexte que celui dans lequel elle avait été demandée.

3.6 Les échanges entre services

Deux services peuvent communiquer de deux manières, et le choix entre les deux n'est pas affaire de goût.

L'**appel direct** convient lorsque l'appelant a besoin de la réponse pour continuer. Il est simple à écrire et à suivre, mais il lie les deux services : si le second est arrêté, le premier échoue.

Le **message** convient lorsque l'appelant n'attend pas de réponse immédiate. Il dépose son message et continue. Le destinataire peut être arrêté ; le message l'attendra.

La règle appliquée dans le projet tient en une phrase : appel direct quand la réponse est nécessaire tout de suite, message dans tous les autres cas. La figure 3.4 montre la topologie qui en résulte.

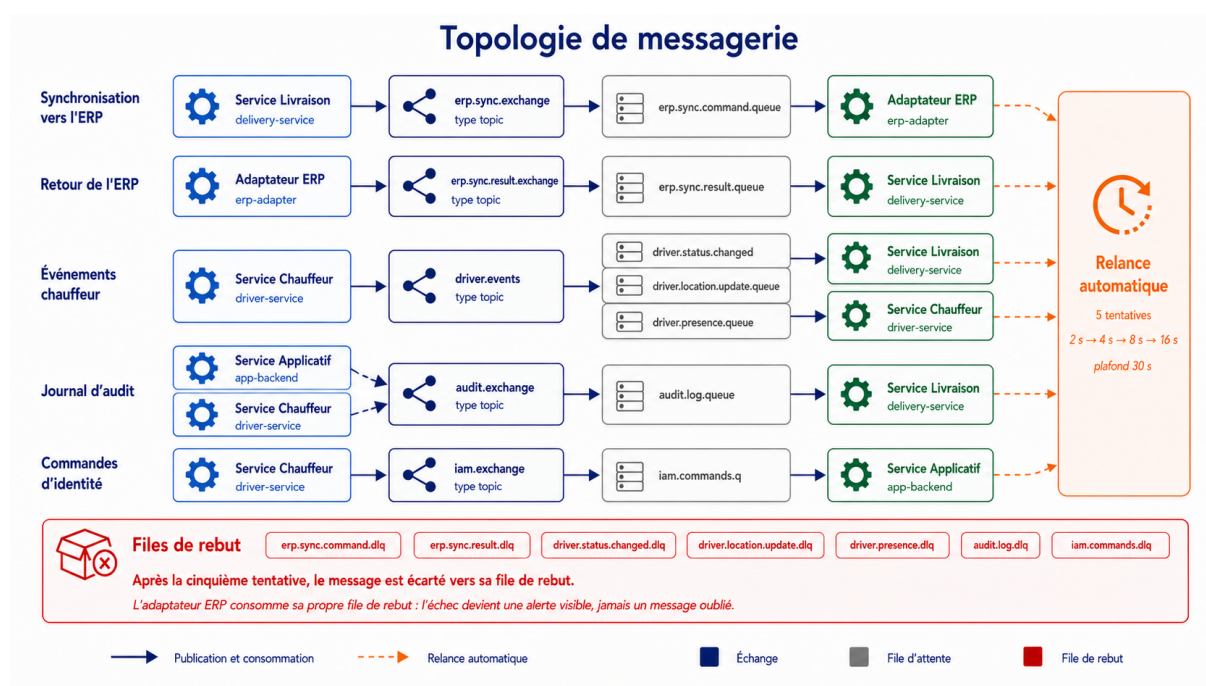


FIG. 3.4 : Topologie de la messagerie

Quatre familles de messages circulent. Les demandes de synchronisation partent du service de livraison vers l'adaptateur, et les résultats reviennent par le chemin inverse. Les événements du chauffeur — changement de statut, position — remontent vers le service de livraison. Les opérations sensibles alimentent le journal d'audit. Les créations de comptes, enfin, sont transmises au service qui pilote le serveur d'identité.

3.6.1 Quand un message ne passe pas

Un message peut échouer parce que le service destinataire est arrêté, parce que la base est indisponible, ou parce que le message lui-même est incorrect. Le traitement est le même dans tous les cas.

Le message est d'abord **réessayé** cinq fois, avec une attente qui double à chaque tentative : deux secondes, puis quatre, huit et seize. Cet espacement croissant laisse à une panne passagère le temps de se résorber, sans saturer un service déjà en difficulté.

Si les cinq tentatives échouent, le message est déplacé dans une **file de rebut**. Il n'est ni perdu ni rejoué indéfiniment : il attend, consultable, que la cause soit corrigée. Sans ce mécanisme, un message définitivement invalide serait repris sans fin et bloquerait tous ceux qui le suivent.

3.7 La mise en marche

L'ensemble est installé par conteneurs. Chaque service, chaque base, chaque composant d'infrastructure est décrit dans un fichier unique qui indique son image, sa configuration et ses dépendances. Une seule commande démarre les dix-sept conteneurs que compte l'installation, répartis comme l'indique le tableau 3.3.

TAB. 3.3 : Les conteneurs de l'installation

Groupe	Nombre	Contenu
Services applicatifs	6	Passerelle, comptes, livraison, chauffeur, assistant, adaptateur ERP
Bases de données	5	Une par service qui en possède une, plus celle du serveur d'identité
Infrastructure	3	Serveur d'identité, bus de messages, stockage d'objets
Moteur d'itinéraires	3	Téléchargement de la carte, préparation, puis exécution

Deux points méritent d'être signalés.

L'**ordre de démarrage** est déclaré, et non espéré. Un service n'est lancé qu'une fois sa base déclarée en bon état ; le serveur d'identité attend la sienne. Sans cette précaution, un service démarrant plus vite que sa base échouerait dès le premier accès.

Les **données survivent aux conteneurs**. Chaque base, le stockage d'objets et le bus écrivent dans un volume nommé, indépendant du conteneur qui l'utilise ; reconstruire une image ne détruit donc aucune donnée.

Ce mode d'installation répond à une exigence posée au chapitre 2 : le système doit pouvoir être installé chez un client sans y installer manuellement une seule dépendance. Le chapitre 9 revient sur son déroulement réel et sur les essais qui l'accompagnent.

3.8 Conclusion

Ce chapitre a fixé la structure dans laquelle les cinq sprints se sont inscrits : six services découpés par domaine métier, chacun maître de sa base ; une passerelle qui vérifie l'identité et les droits avant de transmettre ; une isolation des entreprises clientes obtenue par un schéma dédié plutôt que par une condition à ne pas oublier ; des échanges par message lorsque la réponse peut attendre, avec réessais et file de rebut ; une installation par conteneurs reproductible.

Ces décisions expliquent une bonne part de ce qui suit. Le chapitre 4 met en place l'identité, les droits et l'isolation qui viennent d'être décrits.

4.1 Introduction

4.2 Conclusion

5.1 Introduction

5.2 Conclusion

6.1 Introduction

6.2 Conclusion

7.1 Introduction

7.2 Conclusion

8.1 Introduction

8.2 Conclusion

9.1 Introduction

9.2 Conclusion

Conclusion générale et perspectives

Bibliographie et webographie

- [ASM] ASM TUNISIE. *À propos*. URL : <https://asm-tunisie.com/a-propos/> (visité le 18/08/2026).
- [Onfa] ONFLEET. *Onfleet – Last Mile Delivery Software*. URL : <https://onfleet.com/> (visité le 18/08/2026).
- [Onfb] ONFLEET SUPPORT CENTER. *Driver App Settings*. URL : <https://support.onfleet.com/hc/en-us/articles/10228814951060-Driver-App-Settings> (visité le 18/08/2026).
- [Bria] BRINGG. *Bringg – Delivery and Logistics Orchestration Platform*. URL : <https://www.bringg.com/> (visité le 18/08/2026).
- [Brib] BRINGG HELP CENTER. *Stay Connected with the Bringg Driver App*. URL : <https://help.bringg.com/docs/stay-connected-with-the-bringg-driver-app> (visité le 18/08/2026).
- [Bric] BRINGG HELP CENTER. *Set Up Driver Actions in Order Flows*. URL : <https://help.bringg.com/docs/set-up-driver-actions-in-order-flows> (visité le 18/08/2026).
- [Onfc] ONFLEET. *Onfleet Product Update Q1 2026*. URL : <https://onfleet.com/blog/onfleet-product-update-q1-2026/> (visité le 26/08/2026).
- [Brid] BRINGG. *Predictive Time on Site*. URL : <https://www.bringg.com/> (visité le 26/08/2026).

ANNEXE



Annexes

الخلاصة : تقدّم هذه المذكرة منصّة لتتبع عمليات التوصيل في الكيلومتر الأخير، قابلة للاستعمال من قبل عدّة مؤسسات ومندمجة مع أنظمة تخطيط مواردها. تغطّي المنصّة استيراد الطلبات من نظام تخطيط الموارد، وتخطيط الجولات، والتنفيذ الميداني عبر تطبيق جوال يعمل دون اتصال بالإنترنت، وإثبات التسليم، وإدارة الإرجاع. تعتمد البنية على الخدمات المصغرة مع عزل البيانات حسب المخطط، ومصادقة مفوّضة عبر OAuth2/OIDC، ومُهايّد لأنظمة تخطيط الموارد تم التحقق منه على Odoo وERPNext. تمّت النمذجة باستخدام UML وفق منهجية Scrum.

Résumé : Ce mémoire présente une plateforme de suivi de livraison du dernier kilomètre, exploitable par plusieurs entreprises clientes et intégrée à leurs progiciels de gestion. Elle couvre l'import des commandes depuis l'ERP, la planification des tournées, l'exécution sur le terrain via une application mobile fonctionnant hors connexion, la preuve de livraison et la gestion des retours. L'architecture, orientée microservices, isole les données par schéma, délègue l'authentification en OAuth2/OIDC et intègre un adaptateur ERP agnostique validé sur Odoo et ERPNext. La modélisation a été réalisée en UML et le développement conduit selon la méthode Scrum.

Abstract : This thesis presents a last-mile delivery tracking platform, usable by several client companies and integrated with their enterprise resource planning systems. It covers order import from the ERP, route planning, field execution through an offline-capable mobile application, proof of delivery and returns management. The microservice-oriented architecture isolates data by schema, delegates authentication to OAuth2/OIDC, and integrates an ERP-agnostic adapter validated against both Odoo and ERPNext. Conceptual modelling was carried out using UML and development followed the Scrum agile method.

الكلمات المفاتيح : التوصيل في الكيلومتر الأخير، التتبع الآتي، تعدّد المستأجرين، تكامل أنظمة تخطيط الموارد، الخدمات المصغرة، العمل دون اتصال، UML.

Mots clés : livraison du dernier kilomètre, suivi en temps réel, multi-tenance, intégration ERP, Odoo, ERPNext, microservices, mode hors ligne, UML.

Keywords : last-mile delivery, real-time tracking, multi-tenancy, ERP integration, Odoo, ERPNext, microservices, offline mode, UML.